

IQMS — Simulator Overview

This document describes the accelerated queue simulator implemented under `simulator/`.
It is intended for:

-
- faster-than-real-time scenario testing
 - prediction validation against synthetic workloads
 - UI demos without live camera input
 - seeding the TimescaleDB tables with controlled `SIM_*` data
-

Current Implementation Summary

The simulator stack currently includes:

- a discrete-event flow engine in `simulator/engine.py`
- customer and lane state models in `simulator/entities.py`
- PostgreSQL / TimescaleDB writes and migrations in `simulator/db.py`
- scenario presets in `simulator/scenarios.py`
- calibration from REAL DB data in `simulator/calibrate.py`
- a Streamlit simulator + prediction dashboard in `simulator/app.py`
- a WebSocket simulation engine server in `simulator/server.py`
- an HTML5 Canvas 2D frontend in `simulator/frontend/index.html`
- a self-contained prediction helper module in `simulator/predict_viz.py`
- a smoother local animated viewer in `simulator/pygame_viewer.py`
- launcher commands in `run_simulator.py`

Current status note:

- arrival-side calibration is working against real `entrance_events`
 - service and queue calibration fall back to defaults if `service_events` or `queue_state_snapshots` do not yet contain enough usable REAL rows
 - `predict_viz.py` reads from the database directly; it does not consume in-memory simulator state
 - forecast bucket granularity: **3 minutes** (changed from 5 min, April 2026)
 - `entrance_events` inserts happen at **track death** with entry timestamp, not at first detection (live pipelines only; simulator still inserts at event time)
-

1. Purpose

The simulator models a simplified retail flow:

Entrance (bakery / caddy dwell) → Shop floor → Queue → Caisse / service complete

It does not simulate camera frames. Instead, it uses a discrete-event model so simulated time can advance much faster than wall-clock time while keeping the event order and timestamps logically consistent.

2. Files

File	Purpose
simulator/engine.py	Discrete-event engine using a priority queue (heapq)
simulator/entities.py	Person and Lane` dataclasses
simulator/db.py	PostgreSQL / TimescaleDB writes and schema migrations
simulator/scenarios.py	Scenario presets (normal_day, lunch_rush, evening_rush)
simulator/calibrate.py	Reads REAL DB data and derives simulator defaults
simulator/server.py	Async WebSocket server driving the real-time canvas frontend
simulator/frontend/index.html	HTML5 Canvas 2D store-floor visualisation
simulator/predict_viz.py	Prophet training + Plotly chart builders for the Streamlit prediction tab
simulator/run_batch.py	Batch runner for generating a whole scenario quickly
simulator/test_run.py	Smoke test runner
simulator/app.py	Streamlit dashboard (spawns server.py + HTTP server, embeds canvas)
simulator/pygame_viewer.py	Smooth local Pygame viewer for animated customer flow
run_simulator.py	Unified launcher for batch, smoke test, and dashboard
run_simulator.bat	Windows wrapper for `run_simulator.py

3. Three-Service Architecture

When the Streamlit dashboard runs, it spawns two background processes automatically:

Service	Port	Purpose
Streamlit app	8501	Main UI, controls, prediction tab
WebSocket engine	8080	server.py` – runs the simulation, broadcasts state 30x/sec
HTTP static server	8081	Serves frontend/index.html (the canvas visualisation)

app.py checks each port with `socket.bind()` before spawning to avoid duplicate processes.

The Streamlit iframe embeds `http://localhost:8081/index.html`.

The canvas connects to `ws://localhost:8080` for live updates.

WebSocket message protocol

Client → Server (actions):

Action	Payload fields
start	scenario, lanes, gap, speed, calibrate, persist, hours, start_mode, start_datetime, prob_caddy, prob_bakery
stop	—
update_config	lanes, gap, speed, prob_caddy, prob_bakery` (all optional, live effect)

Server → Client (updates):

Message type	Contents
update	time, entrance[], shop[], lanes[] (with wait_sec), completed
status	message, optionally applied (for config updates) or effective_start/effective_end

4. Core Model

The simulator is event-driven. Event types processed by the engine:

Event	Trigger	Result
ARRIVAL	Exponential inter-arrival gap	Creates a Person, computes entrance dwell, schedules `ENTRANCE_DONE`
ENTRANCE_DONE	After bakery / caddy dwell	Moves person to shop floor, schedules `SHOPPING_DONE`
SHOPPING_DONE	After characteristic-based shop time	Joins shortest queue lane
SERVICE_START	Lane becomes free	Begins service for next queued person
SERVICE_END	After service duration	Logs to DB, starts next person in lane
SNAPSHOT	Every `snapshot_interval_seconds`	Writes queue-state snapshot to DB

Arrivals only fire during open hours (open_hour to close_hour). The engine skips to the next open slot if a scheduled arrival lands outside shop hours.

5. Person Characteristics

Each simulated person is created with:

Field	Description
gender	male / female
age	float 18–80
has_bag	bool — small basket / handbag; shortens shop time slightly
has_caddy	bool — adds 30–90s entrance dwell + 12–28 min shop time
is_group	bool — adds 10–25s entrance dwell + 5–15 min shop time
group_id	string or `None`
picks_bakery	bool — adds 20–90s entrance dwell + 2–6 min shop time
is_express	bool — ~25% of non-caddy customers; shop time 2–5 min

is_express is mutually exclusive with has_caddy (a person with a trolley is never express). The flag is resolved at ARRIVAL and stored on the Person object for use in both shop-time calculation and canvas rendering (■ icon).

These fields are stored in memory and also persisted to entrance_events for simulator-tagged rows.

6. Entrance Dwell Stage

Customers do not go directly from arrival to shop floor. They stop in the **entrance zone** first.

Entrance dwell is computed as:

```
entrance_dwell = (  
    random.uniform(5, 15)                # base pass-through time  
    + (random.uniform(30, 90) if has_caddy else 0) # collect trolley  
    + (random.uniform(20, 90) if picks_bakery else 0) # browse bakery  
    + (random.uniform(10, 25) if is_group else 0) # group coordination  
)
```

During this time the person appears in `people_at_entrance` and is broadcast in the entrance array of the WebSocket update. After dwell completes an `ENTRANCE_DONE` event fires and the person transitions to `people_in_shop`.

The entrance zone in the canvas shows:

- **Bakery display** (top section) — customers with `picks_bakery` cluster here
- **Sliding door** (centre)
- **Caddy rack** (bottom section) — customers with `has_caddy` cluster here

7. Shop-Time Model

Shop time is time-of-day aware. The base duration is selected first:

Condition	Base shop time
<code>is_express</code>	2–5 min
Lunchtime (12–14 h)	4–10 min
Morning (8–10 h)	8–18 min
After-work (17–19 h)	10–20 min
Other hours	5–15 min

Then characteristic modifiers are applied additively:

Characteristic	Effect on shop time
<code>has_caddy</code>	+12–28 min
<code>has_bag`</code> (no caddy)	–6—2 min (small basket → shorter trip)
<code>is_group</code>	+5–15 min
<code>is_group + has_caddy</code>	+5–10 min (family weekly shop bonus)
<code>picks_bakery</code>	+2–6 min
<code>age > 65</code>	+5–12 min
<code>age < 25</code>	–4–0 min
<code>35 ≤ age ≤ 55</code>	0–5 min (prime full-shop age)

The final shop time is clamped with `max(2, ...)` so it never drops below 2 minutes.

8. Queue and Service Model

After shopping is complete:

- The person joins the shortest queue (by total load = queue length + 1 if serving)
- If the chosen lane is free, `SERVICE_START` fires immediately

- Service duration:
 - `base_service_seconds` (default 30s)
 - +20s if `has_bag`
 - +60s if `has_caddy`
 - +30s if `is_group`
 - random noise 0–30s

The server estimates per-lane wait time using `_lane_wait_sec()`, which sums remaining service time for the current customer plus estimated service for all queued customers.

Lane count can be changed live via the `update_config` WebSocket action without restarting the simulation. The engine rebalances queues automatically when lanes are added or removed.

9. Shop Open Hours

The engine respects configurable open/close hours:

```
open_hour = config.get('open_hour', 8)
close_hour = config.get('close_hour', 20)
```

- Arrivals are only scheduled during open hours.
- If a computed next-arrival lands outside open hours, it is bumped to `open_hour` the next day.
- `predict_viz.py` applies the same constants for forecast training (zero-fills closed hours) and forecast generation (only produces predictions during open hours).

10. Database Integration

Simulator rows are tagged with:

```
camera_id = SIM_<scenario>
```

This allows real and simulated rows to coexist in the same database while remaining filterable by source.

Tables written by the simulator

- `entrance_events`
- `queue_state_snapshots`
- `service_events`

Persisted simulator-specific fields

`entrance_events`:

- `has_bag`, `has_caddy`, `is_group`, `group_id`

`queue_state_snapshots`:

- `max_dwell_sec`

`service_events`:

- `lane_id`

Migration style

Schema upgrades in `simulator/db.py` use safe, idempotent `DO $$ ... IF NOT EXISTS (...) THEN ALTER TABLE ... END $$` blocks rather than destructive schema resets.

11. Prediction Integration

Both forecast scripts support source filtering:

```
python prophet_predict.py --source SIM
python ensemble_predict.py --source SIM
```

Supported values: REAL, SIM, ALL

`predict_viz.py` (used in the Streamlit prediction tab):

- Fills closed hours with explicit zeros so Prophet learns the open/close pattern
 - Only generates future predictions for open-hour timestamps
 - Reads queue history from `queue_state_snapshots` regardless of REAL/SIM source filter
-

12. Canvas Frontend

The canvas visualisation (`simulator/frontend/index.html`) renders a top-down 2D view of the store:

Zone	Content
Entrance panel (left)	Bakery display, sliding door, caddy rack
Shop floor	3 merchandise aisles (Fruits & Veg, Dairy & Cold, Beverages), serpentine customer path
Checkout section (right)	Per-lane boxes with wait-time indicator and queue

Customer entities are rendered as emoji (■■■■■■■■) inside a coloured ring:

Ring colour	Meaning
Green	At entrance (bakery / caddy / passing through)
Blue / pink	Shopping on floor
Amber	Waiting in queue
Red	Being served

An item badge (■ express / ■ caddy / ■ bakery / ■ bag) appears at the top-right of the entity.

A dwell/wait pill badge appears below the entity with the elapsed time for the current stage.

An activity badge (BAKERY / CADDY / ENTERING) floats above entrance-zone customers.

13. How To Run

Install simulator dependencies

From the Queue-Management folder:

```
python -m pip install -r simulator/requirements.txt
```

Recommended launcher

```
python run_simulator.py calibrate
python run_simulator.py calibrate --days 30 --output json
python run_simulator.py batch normal_day
python run_simulator.py batch lunch_rush
python run_simulator.py batch evening_rush
python run_simulator.py test
python run_simulator.py dashboard
python run_simulator.py viewer normal_day
python run_simulator.py viewer normal_day --calibrate
```

Windows wrapper

```
run_simulator.bat calibrate --days 30 --output report
run_simulator.bat batch normal_day
run_simulator.bat test
run_simulator.bat dashboard
run_simulator.bat viewer normal_day
```

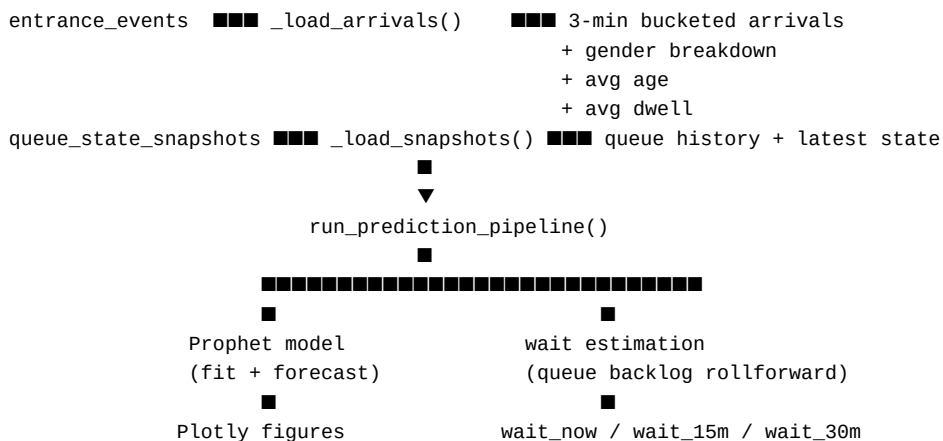
Direct entry points

```
python -m simulator.calibrate
streamlit run simulator/app.py
streamlit run simulator/app.py -- --calibrate --days 30
python simulator/pygame_viewer.py --calibrate
```

14. Prediction Tab — `predict_viz.py`

The Streamlit dashboard includes a full prediction tab powered by `predict_viz.py`. It is independent of the simulator engine and reads directly from the live database.

Data pipeline



Bucket granularity

Arrivals are aggregated into **3-minute buckets** via `time_bucket('3 minutes', timestamp)`. This is the unit Prophet trains on and forecasts at.

Closed hours are filled with explicit `y=0` rows so Prophet learns the open/close pattern (`_add_closed_zeros`). Future forecast timestamps are generated only for remaining open hours today (`_future_open_timestamps`).

Bucket fields (since April 2026)

Each bucket now carries enriched aggregates beyond entry count:

Field	Source	Notes
entry_count	COUNT(*)	total arrivals
male_count	COUNT(*) FILTER (WHERE gender='male')	Pipeline B only
female_count	COUNT(*) FILTER (WHERE gender='female')	Pipeline B only
avg_age	AVG(age_estimate)	Pipeline B only
avg_dwell_sec	AVG(dwell_seconds) FILTER (WHERE dwell_seconds > 0)	both pipelines

Live actuals

`load_today_actuals()` is called on every Streamlit render (bypasses the training cache) so the blue dots on the forecast chart always reflect the current DB state even while the Prophet model result is still cached.

Retrain scheduling

The prediction cache is managed via `st.session_state` with a configurable TTL (1–60 min slider in the sidebar). A **Force Retrain** button invalidates the cache immediately. The trained-at / next-retrain times are shown below the forecast chart.

Data quality filters

- **data_since** date filter — exclude data before a chosen date (useful when DB contains historical pollution)
- **Outlier cap** — clips any bucket exceeding max(99th percentile of non-zero buckets, 30) before fitting, preventing isolated spikes from warping the model

15. Camera Diagnostics Expander

The Streamlit dashboard includes a **Camera ID diagnostics** expander (collapsed by default) that provides:

- **Entries per camera_id** — total row count and latest entry per camera in `entrance_events`
- **Top spike buckets** — highest 3-min bucket counts with camera breakdown; warns if any real camera exceeds 100 entries/bucket
- **Fix SIM annotations** — renames known bad SIM camera IDs (without `SIM_` prefix) in both `entrance_events` and `queue_state_snapshots`
- **Duplicate restart cleanup** — automatically runs on open: deletes entries with `NULL dwell_seconds` (brief re-detections) from real cameras and reports how many were removed

16. Current Limitations

- The simulator models logical flow, not camera-frame detection behavior.
- There is no separate explicit `EXIT` event; flow ends at `SERVICE_END`.
- `queue_state_snapshots` are global per simulator camera, not per lane.

- The simulator writes into shared production tables using `SIM_*` camera IDs rather than separate simulator-only tables.
 - `track_id` is session-local and resets on simulator restart.
 - `service_events` exists but is not yet automatically populated by the live production pipelines (only by the simulator).
 - Gender/age bucket enrichment is only meaningful for Pipeline B (`Bosch_Camera_Entrance`); Pipeline A always stores `unknown/NULL`.
-

17. Recommended Use

Use the simulator when you want to:

- generate a day of synthetic activity quickly
- compare REAL vs SIM forecast behavior
- validate queue and wait estimation logic
- demo the full store-floor flow visually without RTSP input

For production camera validation, use the live pipelines instead of the simulator.